
# Natural Language Processing ( Spring-2026 )

## Assignment-2

## Instructor Dr. Hajra Waheed

## Teacher Assistant

- **Abdur Rehman:** abdurrehman.ta2025@lhr.nu.edu.pk
- For **assignment & Viva-related queries**, please contact the TA directly via email.

### Submission Guidelines:

- Submit your assignment on Google Classroom in the format "20XX.ipynb".
- **The deadline is April 5, 2026, at 11:59 PM. No extensions will be granted.**

### Declarations:

- Late submissions will incur penalties: 25% deduction on the first day, 50% on the second day, and zero marks thereafter.
- **Plagiarism will result in zero marks for the assignment.**
- This is an individual assignment; collaboration or group work is strictly prohibited.
- Please ensure that you submit your own original work.

### VIVA Policy:

- A VIVA (oral examination) will be conducted to assess your understanding of the assignment.
- The VIVA will be scheduled separately, and you will be notified of the date and time.
- Failure to attend the VIVA will result in zero marks for the assignment.

### Academic Integrity:

- Plagiarism, collusion, and academic dishonesty will not be tolerated.
- Any instances of academic misconduct will be reported to the authorities and may result in severe penalties.

# Objective

The objective of this assignment is to develop a practical and conceptual understanding of **sequence modeling architectures used in modern NLP systems**, including **Recurrent Neural Networks (RNN), LSTM, GRU, Seq2Seq models, Encoder–Decoder architectures, and Attention mechanisms**.

Students will implement and analyze neural sequence models for **machine translation**, comparing different architectures and evaluating their performance using standard NLP metrics.

NOTE:

- USE GPU for Faster Training
- Students are encouraged to train models for a reasonable number of epochs (typically 10–30 epochs) depending on available computational resources.

# Part 1: Sequence Modeling with Recurrent Neural Networks (20%)

This part introduces basic sequence modeling using RNNs and examines their ability to learn temporal dependencies in text. It focuses on how sequential information is processed step-by-step, allowing the model to retain contextual knowledge from previous tokens while predicting the next element in a sequence. Through this exploration, students will gain practical insight into how recurrent architectures capture linguistic patterns and dependencies in natural language data.

## Dataset

**WikiText-2-v!**

- Source: HuggingFace datasets
- https://huggingface.co/datasets/Salesforce/wikitext/viewer/wikitext-2-v1
- Domain: Wikipedia articles

## Tasks

### Task:P1T1. Dataset Preprocessing

Students must:

1. Load the WikiText-2-v1 dataset
2. Normalize the text corpus
3. Perform tokenization
4. Build vocabulary

Report:

- Vocabulary size
- Total tokens
- Average sentence length

### Task:P1T2. Implement Vanilla RNN Language Model

Students must implement an **RNN language model**.

Design your Architecture and Train the model to **predict the next word in a sequence**.

Report:

- Training loss
- Validation loss
- Perplexity

### Task:P1T3. Sequence Generation

Use the trained RNN model to generate sequences.

Example:

> Input: Natural language
> Output: Natural language processing enables machines to understand text

Deliverables:

- Generate **20 sequences**
- Discuss sequence quality and choose evaluation metric and why you choose this.

# Part 2: LSTM vs GRU Sequence Models (20%)

This part explores gated recurrent architectures designed to handle long-term dependencies in sequential data. Traditional recurrent models often struggle with vanishing and exploding gradient problems, which limit their ability to capture relationships across long sequences. To address these limitations, more advanced recurrent architectures introduce gating mechanisms that regulate the flow of information through the network.

Students will implement two gated architectures:

- **Long Short-Term Memory (LSTM)**
- **Gated Recurrent Unit (GRU)**

The objective is to **build language models that predict the next word in a sequence** and compare the effectiveness of these architectures.

### Important Implementation Rule

Students **must implement LSTM and GRU manually**.

The following built-in implementations are **NOT allowed**:

- `nn.LSTM`
- `nn.GRU`
- `torch.nn.LSTMCell`
- `torch.nn.GRUCell`

Students must implement the **gating equations and hidden state updates themselves** using basic tensor operations.

Allowed components:

- Embedding layers

- Linear layers
- Activation functions
- Optimizers

## Dataset

**WikiText-2-v!**

- Source: HuggingFace datasets
- https://huggingface.co/datasets/Salesforce/wikitext/viewer/wikitext-2-v1
- Domain: Wikipedia articles

## Tasks

### Task:P2T1. Dataset Preparation

Students must prepare the dataset for sequence modeling.

Required steps:

- Load the dataset from Hugging Face.
- Convert text to lowercase.
- Tokenize sentences into words.
- Build a vocabulary dictionary.
- Replace rare words with `<UNK>` tokens.
- Add `<SOS>` and `<EOS>` tokens.

Students must create **input–target training pairs** for next-word prediction.

Example:

| Input Sequence | Target |
|---|---|
| the | cat |
| the cat | sat |
| the cat sat | on |

Students must define a **fixed sequence length** for training samples.

### Task:P2T2. Implement LSTM Cell from Scratch

Students must implement a **custom LSTM cell**.

The implementation must include:

- input gate
- forget gate
- output gate
- candidate cell state
- hidden state update
- cell state update

Students must construct a **complete LSTM language model** using the custom LSTM cell.

The model must be trained to predict the **next word in a sequence**.

Students must monitor:

- training loss
- validation loss

## Task:P2T3. Implement GRU Cell from Scratch

Students must implement a **custom GRU cell**.

The implementation must include:

- update gate
- reset gate
- candidate hidden state
- hidden state update

Students must construct a **GRU language model** using the custom GRU cell.

The same dataset and preprocessing pipeline must be used as in Task P2T2.

Students must monitor:

- training loss
- validation loss

## Task:P2T4 . Model Training

Students must train both models using identical training settings.

Students must record:

- training loss curves
- validation loss curves
- total training time

Students must train each model for **multiple epochs** until convergence.

## Task:P2T5. Next Word Prediction

Students must demonstrate predictions generated by the trained models.

Students must provide **10 examples of next-word predictions**.

For each example report:

| Input Sequence | Actual Word | Predicted Word |
|---|---|---|

### Task:P2T6. Model Evaluation

Students must evaluate models using **perplexity**, which measures how well a language model predicts sequences.

Students must compute:

- validation perplexity
- test perplexity

Students must also record **training time for each model**.

### Task:P2T7. Gate Behavior Analysis

Students must analyze the behavior of gates inside the models.

Students must select **one validation sentence** and record the **average activation values** of gates.

Example:

| Model | Gate | Average Activation |
|---|---|---|
| LSTM | Input Gate | |
| LSTM | Forget Gate | |
| LSTM | Output Gate | |
| GRU | Update Gate | |
| GRU | Reset Gate | |

Students must briefly explain how gates control **information flow in recurrent networks**.

### Task:P2T8. Long-Context Prediction Test

Students must test the models on **longer sequences (≥30 tokens)**.

Students must compare predictions from:

- RNN
- LSTM
- GRU

Example table:

| Model | Predicted Word | Confidence |
|---|---|---|
| RNN | | |
| LSTM | | |
| GRU | | |

Students must analyze how well each model **handles long-context dependencies**.

## Final Model Comparison

Students must compare all three models:

| Model | Validation Perplexity | Test Perplexity | Training Time |
|---|---|---|---|
| RNN (Part 1) | | | |
| LSTM (Custom) | | | |
| GRU (Custom) | | | |

Students must discuss:

- which model performs best
- whether gated architectures improve sequence learning
- structural differences between LSTM and GRU
- advantages and limitations of each model

# Part 3: Seq2Seq Encoder–Decoder Model (20%)

This part introduces **sequence-to-sequence (Seq2Seq) models** for machine translation, where an input sentence in one language is transformed into an output sentence in another language. The model is based on an **encoder–decoder architecture**: the encoder reads the source sentence and converts it into a context representation, while the decoder generates the translated sentence one token at a time.

Students will implement a **basic Seq2Seq translation system** to translate **English sentences into German**.

## Dataset

**Multi30k English–German Dataset**
Source: Hugging Face Datasets
Dataset: `bentrevett/multi30k`

https://huggingface.co/datasets/bentrevett/multi30k

Example:

- **English:** A child is playing with a ball
- **German:** Ein Kind spielt mit einem Ball

Students may use a **subset of the dataset** if computational resources are limited, but the same subset must be used consistently across all tasks in this part.

## Important Implementation Rules

Students must implement a **Seq2Seq encoder–decoder model** for translation.

**Allowed:**

- built-in recurrent layers such as **LSTM** or **GRU** for encoder and decoder
- embedding layers
- linear output layers
- optimizers and loss functions

**Not allowed:**

- pre-trained translation models
- Hugging Face Trainer for direct training
- Transformer-based translation models in this part

## Tasks

### Task:P3T1. Implement Seq2Seq Model

Students must implement a **basic encoder–decoder architecture** for neural machine translation.

## Required pipeline

1. Source sentence
2. Encoder
3. Context vector
4. Decoder
5. Translated sentence

## Requirements

- The **encoder** must process the English source sentence.
- The **decoder** must generate the German translation token by token.
- Students may use **LSTM or GRU** for the encoder and decoder.
- The model must include:
    - source vocabulary
    - target vocabulary
    - special tokens such as `<SOS>`, `<EOS>`, `<PAD>`, and `<UNK>`
- Students must clearly define:
    - input representation
    - output representation
    - stopping condition for decoding

### Task:P3T2. Translation Generation

Students must generate translations for **20 test sentences** using the trained Seq2Seq model.

For each example, students must report:

| Source Sentence | Reference Translation | Model Prediction |
|---|---|---|

The examples must be taken from the **test split** of the dataset.

### Task:P3T3.Basic Evaluation

Students must evaluate translation quality using:

- **BLEU score**
- **token-level accuracy**

Students must report the evaluation results on the **test set**.

### Task:P3T4. Error Analysis

Students must analyze **10 incorrect or weak translations** produced by the model.

Students should identify common problems such as:

- incorrect word order
- missing words
- repeated words
- incomplete translation
- wrong lexical choice

Students must briefly explain why these errors may occur in a basic Seq2Seq model without attention.

# Part 4: Attention Mechanism in Translation (20%)

This part improves translation performance by introducing **attention mechanisms** into the sequence-to-sequence architecture. In traditional encoder–decoder models, the entire source sentence is compressed into a single fixed-length **context vector**, which can cause **information loss**, particularly for longer sentences.

Attention mechanisms solve this limitation by allowing the **decoder to dynamically focus on relevant parts of the source sentence during translation generation**.

Instead of relying on a single context representation, the decoder computes **attention weights over encoder hidden states** and generates a context vector for each decoding step

.

## Dataset

### WMT14 English-German Translation Dataset

https://huggingface.co/datasets/wmt/wmt14/viewer/de-en

Students must use **500K–1M sentence pairs from a train subset** for training and testing from test/val subset

## Tasks

### Task:P4T1. Implement Attention Mechanisms

Students must extend the  Any **Seq2Seq model from Part 3** by implementing **three attention mechanisms**.

Required attention mechanisms:

1. **Bahdanau Attention (Additive Attention)**
2. **Luong Attention (Multiplicative Attention)**
3. **Scaled Dot-Product Attention**

Students must implement the attention mechanism that computes **attention scores between decoder hidden states and encoder hidden states** and produces a **context vector** used by the decoder.

## Model Architecture

1. Source sentence
2. Encoder
3. Encoder hidden states
4. Attention mechanism
5. Context vector
6. Decoder
7. Translated sentence

Students must explain:

- how attention scores are computed
- how attention weights are normalized
- how the context vector is combined with decoder states

## Task:P4T2. Attention Visualization

Students must visualize **attention weights** produced during translation.

For **at least 15 translation examples**, students must generate **attention alignment heatmaps**. Use matplotlib or seaborn to visualize attention weights.

Example:

| English | German |
|---|---|
| I love machine learning | Ich liebe maschinelles Lernen |

Students must present:

- attention heatmap
- alignment between source and target tokens

Students must interpret the visualization by explaining:

- which source words influence each predicted target word
- whether the alignment is linguistically meaningful

## Task:P4T3. Attention Model Comparison

Students must compare the following models:

| Model | BLEU Score |
|---|---|
| Seq2Seq (No Attention) | |
| Seq2Seq + Bahdanau Attention | |

| | |
|---|---|
| Seq2Seq + Luong Attention | |
| Seq2Seq + Scaled Dot Attention | |

Students must analyze:

- which attention mechanism performs best
- how attention improves translation quality
- differences between additive and multiplicative attention
- whether attention helps with longer sentences

Students must include:

1. Implementation of **three attention mechanisms**
2. Attention heatmaps for **at least 15 examples**
3. Translation results for each attention model
4. A comparison table of attention models
5. Short discussion of results

# Part 5: Neural Machine Translation System (20%)

This part requires students to train a complete Neural Machine Translation (NMT) system and compare different model variants developed in earlier parts of the assignment. Students will integrate the encoder–decoder architecture, recurrent sequence models, and attention mechanisms to build a full translation pipeline capable of converting source sentences into target language translations.

### Dataset

### WMT14 English-German Translation Dataset

https://huggingface.co/datasets/wmt/wmt14/viewer/de-en

Students must use **500K–1M sentence pairs from a train subset** for training and for testing usetest/val subset

Example:

English: The project was successful.
German: Das Projekt war erfolgreich.

### Tasks

### Task:P5T1. Train Translation Models

Students must train the following models:

| Model | Encoder | Decoder | Attention |
|---|---|---|---|
| Seq2Seq RNN | RNN | RNN | No |
| Seq2Seq LSTM | LSTM | LSTM | No |
| Seq2Seq GRU | GRU | GRU | No |
| Seq2SeqLSTM + AttentionVariants | LSTM | LSTM | YES/Name |
| Seq2SeqGRU + AttentionVariants | GRU | GRU | YES/Name |

## Task:P5T2. Translation Evaluation

Students must evaluate translation performance using the following metrics.

### BLEU Score

Measures n-gram overlap between reference and prediction.

### BERTScore

Measures **semantic similarity using contextual embeddings** from BERT.

This metric captures meaning similarity even when wording differs.

Students must compute:

- BLEU
- BERTScore
- METEOR

### Final Model Comparison Table

Students must produce the following comparison table.

| Model | BLEU | BERTScore | Sentence Accuracy | METEOR | Training Time |
|---|---|---|---|---|---|
| Seq2Seq RNN | | | | | |
| Seq2Seq LSTM | | | | | |
| Seq2Seq GRU | | | | | |
| LSTM + AttentionVariants | | | | | |
| GRU + AttentionVariants | | | | | |

## Task:P5T3. Translation Examples

Students must generate **translation examples based on the last two digits of their roll number**.
For example:

- If the roll number is **8023**, students must generate translation examples in three batches: **8, 2, and 3**.
- If the roll number is **9015**, students must generate translation examples in three batches: **9, 1, and 5**.

For **each example**, students must report the following:

1. **Source sentence** (original sentence in the source language)
2. **Reference translation** (the correct human translation)
3. **Model prediction** (the translation generated by the model)

## Task:P5T4. Error Analysis

Students must analyze **10 incorrect translations**.

Possible issues:

| Error Type | Description |
|---|---|
| Word order error | incorrect German syntax |
| Missing words | incomplete translation |
| Repeated tokens | repeated outputs |
| Rare word errors | unknown token translation |

Students must explain:

- why the error occurred
- how attention improved results

# Deliverables

1. **1 Notebook**
    - A single notebook containing code implementations for all parts, organized sequentially by the specified TASK IDs.
    - All results must be visible and reproducible, with every cell already executed.
    - The notebook must not require re-running to verify outputs.
2. **1 Detailed Report**
    - A well-structured written report covering each task separately.
    - Include screenshots of results and outputs for every task.
    - Generated translations
    - Evaluation metrics
    - Attention visualizations
    - Model architecture diagrams
    - Experimental results
    - Translation examples
    - Error analysis
    - Ensure proper academic formatting, clear headings, and consistent labeling.
3. **Exported Files**
    - All required exported artifacts (e.g., generated data files, plots, models, or result files) referenced in the notebook and report.

Screenshots in the report should correspond to key results and tables shown in the notebook.

### Important Notes

- Incomplete execution, missing TASK IDs, or improperly formatted reports may result in loss of marks.
- Ensure consistency between notebook outputs, screenshots, and exported files.